Now we're talking — this is a **serious system design shift**.

You've just made 2 important architectural decisions:

1. ❌ No Clerk → you control auth (more flexibility, more responsibility)
2. ✅ Guest checkout → lower friction → **higher conversions**

So I'm rewriting your blueprint as a **clean, production-ready system (v4)** with:

- Custom auth
- Guest + user hybrid system
- Multi-admin management
- Fully connected flows

🧠 AI TRAVEL OPERATING SYSTEM — FINAL TECHNICAL BLUEPRINT (v4)

---

1. 🏗️ SYSTEM ARCHITECTURE

4 Core Layers

1. Frontend (Next.js 14)

- App Router
- Server Components → SEO pages
- Client Components → dashboards, forms, AI UI

---

2. Backend (Convex)

- Database + logic + real-time
- No REST API needed

---

3. AI Layer

- Groq API via Convex Actions
- Structured JSON responses

---

4. External Services

- Email (Resend)
- WhatsApp (Phase 2)

---

2. 🔐 AUTH SYSTEM (CUSTOM + GUEST SUPPORT)

AUTH STRATEGY

We use **hybrid auth**:

1. Registered Users

- Email + password (or OTP later)
- Have dashboard access

2. Guest Users (IMPORTANT)

- Can book WITHOUT account
- Stored as “guest bookings”
- Can later convert to full account

SESSION MANAGEMENT

Use:

- HTTP-only cookies
- JWT (or Convex session table)

USER STATES

Type	Can Book	Dashboard	Saved Data

Guest	✅	❌	Minimal
Customer	✅	✅	Full
Admin	✅	✅	Full
SuperAdmin	✅	✅	Full

---

3. 🧩 DATABASE DESIGN (UPDATED)

---

users

```
{
  _id,
  name,
  email,
  phone,
  passwordHash,
  role, // "super_admin" | "admin" | "customer"
  createdAt
}
```

---

guestBookings (NEW)

```
{
  _id,
  name,
  phone,
  email (optional),
  tourId,
  peopleCount,
  notes,
  status, // pending | confirmed | cancelled
  createdAt
}
```

---

bookings (REGISTERED USERS)

```
{
```

```
  _id,
  userId,
  tourId,
  peopleCount,
  totalPrice,
  status,
  notes,
  createdAt
}
```

---

tours

```
{

  _id,

  title,

  slug,

  description,

  price,

  durationDays,

  location,

  images: string[],

  itinerary: {

    day: number,

    title: string,

    description: string

  }[],

  isActive,

  createdAt

}
```

---

leads

```
{
  _id,
  name,
  phone,
  source, // AI | Booking | Manual
  message,
  createdAt
}
```

---

aiRequests

```
{

  _id,

  userId?,

  input,

  output,

  type,

  createdAt

}
```

---

blogPosts

```
{

  _id,

  title,

  slug,

  content,
```

```
  metaTitle,

  metaDescription,

  published,

  createdAt

}
```

---

adminLogs (NEW 🔥)

```
{
  _id,
  action,
  performedBy,
  targetUser,
  timestamp
}
```

---

4. 🔌 CONVEX FUNCTION ARCHITECTURE

AUTH

- `registerUser`
- `loginUser`
- `getCurrentUser`

ADMIN MANAGEMENT (NEW CORE SYSTEM)

- `createAdmin`
- `promoteUser`
- `demoteAdmin`

- `getAdmins`
- `getUsers`

TOURS

- `getTours`
- `getTourBySlug`
- `createTour`
- `updateTour`
- `deleteTour`

BOOKINGS

Registered

- `createBooking`
- `getUserBookings`

Guest

- `createGuestBooking`

ADMIN BOOKINGS

- `getAllBookings`
- `updateBookingStatus`

LEADS

- `createLead`
- `getLeads`

AI

- `generateTrip`
- `chatAssistant`

BLOG

- `createPost`
- `getPosts`

---

5. ⚡ CORE SYSTEM FLOWS

🧳 BOOKING FLOW (DUAL SYSTEM)

Guest Flow (HIGH PRIORITY)

User → Tour Page
→ Click Book Now
→ Fill form (name, phone, people)
→ createGuestBooking()
→ status = pending
→ Lead captured
→ Admin notified

Registered User Flow

User logged in
→ createBooking()
→ booking linked to userId
→ appears in dashboard

🔁 CONVERSION FLOW (IMPORTANT)

Guest books
→ Later creates account with same phone/email
→ System links old bookings

🤖 AI PLANNER FLOW

User inputs query
→ Convex action
→ Fetch tours
→ Groq call
→ Return structured plan
→ Save lead

🧑‍💼 ADMIN FLOW

Admin login
→ Dashboard access
→ Manage:
  tours
  bookings
  users
  admins

---

6. 🔐 AUTHORIZATION LOGIC

ROUTE PROTECTION

/admin → admin OR super_admin
/admin/manage-admins → ONLY super_admin
/dashboard → logged-in users

BACKEND VALIDATION

Always check:

```
if (!user) throw Error
if (role !== "admin") block
```

---

7. 🧑‍💻 ADMIN PANEL (UPGRADED)

CORE SECTIONS

- Tours
- Bookings (Guest + User unified view)
- Users
- Blog
- Analytics
- **Manage Admins** 🔐

BOOKINGS TABLE (IMPORTANT)

Unified view:

Type	Name	Tour	Status
Guest	Ali	Hunza	Pending
User	Sara	Skardu	Confirmed

MANAGE ADMINS

- Promote user → admin
- Demote admin
- View all admins

---

8. 🎨 UI STRUCTURE

PUBLIC

- Homepage → AI planner hero
- Tour page → conversion focused

- Booking modal (fast)

DASHBOARD

- Booking history
- Status tracking

ADMIN UI

- Sidebar layout
- Tables + filters
- Real-time updates

---

9. 🔥 SEO ENGINE

CORE PAGES

- /tours/[slug]
- /destinations/[slug]
- /blog/[slug]

PROGRAMMATIC SEO

Auto pages:

- /hunza-trip-from-lahore
- /skardu-tour-cost

METADATA + SCHEMA

- Titles + descriptions
- Tour schema

- FAQ schema

SITEMAP

Auto-generated

---

10. 📡 NOTIFICATIONS

PHASE 1

- Email (Resend)
- WhatsApp click link

PHASE 2

- WhatsApp API
- Auto triggers via Convex

---

11. 🚀 DEPLOYMENT

Frontend:

- Vercel

Backend:

- Convex

Flow:

Client → Next.js → Convex → DB

→ AI

---

12. ⚡ BUILD ROADMAP

🔥 PHASE 1 (MVP – 3–5 days)

- Tours pages (SEO)
- Guest booking system
- Admin panel (basic)
- Auth (simple login)

👉 This alone can CLOSE your client

⚡ PHASE 2

- User accounts + dashboard
- Admin management system
- AI planner

🚀 PHASE 3

- Blog CMS
- WhatsApp automation
- Analytics

---

13. 🧠 CRITICAL ENGINEERING DECISIONS

---

1. Guest Checkout FIRST

Removes friction → increases bookings

---

2. Admin Panel = Core Product

Client cares about control, not code

---

3. AI = Differentiator, not foundation

Don’t block MVP on AI

---

4. Keep Everything Real-time

Convex advantage → use it everywhere